Module 06 Lab - Regression and Classification Models

Objective: To understand the difference between regression and classification and to build your first linear models for both tasks.

In this lab, you will write the code to train and evaluate the models.

Part 1: Regression vs. Classification

This is the most fundamental distinction between types of supervised learning problems.

- **Regression:** The goal is to predict a **continuous numerical value**.
  - *Examples:* Predicting the price of a house, the temperature tomorrow, or the stock price.
- **Classification:** The goal is to predict a **discrete category or class label**.
  - *Examples:* Predicting if an email is spam or not spam, if a flower is a setosa, versicolor, or virginica, or if a customer will churn or not.

In this lab, we will tackle one of each.

Part 2: Linear Regression

Concept: Linear Regression is used to predict a continuous value. It works by finding the best-fitting straight line through the data points. The model learns a "slope" (coefficient) for each feature and an "intercept".

- **Problem:** We will predict the `Fare` of a Titanic passenger based on their `Age` and `Pclass`.

In [1]:
```
# Import pandas to work with the dataset
import pandas as pd

# Import NumPy for mathematical operations, including square root
import numpy as np

# Import the function used to split data into training and testing sets
from sklearn.model_selection import train_test_split

# Import the Linear Regression model
from sklearn.linear_model import LinearRegression

# Import Mean Squared Error to evaluate regression predictions
```

```
from sklearn.metrics import mean_squared_error

# Load the Titanic dataset from the provided URL
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets

# Remove rows where Age is missing
df.dropna(subset=['Age'], inplace=True)

# Define the input features used to predict the fare
features = ['Age', 'Pclass']

# Define the target variable we want to predict
target_reg = 'Fare'

# Store the input features in X_reg
X_reg = df[features]

# Store the target values in y_reg
y_reg = df[target_reg]

# Split the data: 80% for training and 20% for testing
# random_state=42 makes the split reproducible
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y
```

Task 1: Train and Evaluate a Linear Regression Model

Your Task:

1. Create an instance of the `LinearRegression` model.
2. Train the model using the training data (`X_train_reg` , `y_train_reg`).
3. Make predictions on the test data.
4. Evaluate the model using `mean_squared_error` . This metric tells us the average of the squared differences between the predicted and actual values.

In [2]:
```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# LinearRegression creates a model that predicts numerical values
lr_model = LinearRegression()

# 2. Train the model
# The model learns the relationship between Age, Pclass, and Fare
lr_model.fit(X_train_reg, y_train_reg)

# 3. Make predictions
# Use the test features to predict passenger fares
y_pred_reg = lr_model.predict(X_test_reg)

# 4. Evaluate the model
# MSE measures the average squared difference between actual and predicted f
mse = mean_squared_error(y_test_reg, y_pred_reg)

# Print the Mean Squared Error
```

```
print(f"Mean Squared Error for Fare Prediction: {mse:.2f}")

# Calculate the square root of MSE to get RMSE
# RMSE is in the same units as Fare, which is dollars
print(
    f"The square root of this is {np.sqrt(mse):.2f}, "
    "meaning this is the model's typical prediction error in dollars."
)

# Display the coefficients for Age and Pclass
# Their order matches the features list: Age first, then Pclass
print("Linear Regression Coefficients:", lr_model.coef_)

# Display the intercept learned by the model
print("Linear Regression Intercept:", lr_model.intercept_)
```

```
Mean Squared Error for Fare Prediction: 3364.92
The square root of this is 58.01, meaning this is the model's typical predic
tion error in dollars.
Linear Regression Coefficients: [ -0.43695889 -38.29728935]
Linear Regression Intercept: 133.30726668128784
```

Part 3: Logistic Regression

Concept: Despite its name, Logistic Regression is a **classification** algorithm. It works by calculating the probability that a given input belongs to a certain class. It's one of the most widely used and interpretable classification models.

- **Problem:** We will predict whether a passenger `Survived` based on their `Age`, `Pclass`, and `Sex`.

In [3]:
```
# Import Logistic Regression for classification
from sklearn.linear_model import LogisticRegression

# Import accuracy_score to evaluate classification predictions
from sklearn.metrics import accuracy_score

# Prepare data for classification
# Convert Sex into numerical values because the model needs numerical inputs
# Male = 0 and Female = 1
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

# Define the input features for the classification model
features_cls = ['Age', 'Pclass', 'Sex']

# Define the target variable we want to predict
target_cls = 'Survived'

# Store the input features in X_cls
X_cls = df[features_cls]

# Store the actual survival labels in y_cls
# 0 = did not survive; 1 = survived
y_cls = df[target_cls]
```

```
# Split the data into training and testing sets
# 80% for training and 20% for testing
X_train_cls, X_test_cls, y_train_cls, y_test_cls = train_test_split(X_cls, y
```

Task 2: Train and Evaluate a Logistic Regression Model

Your Task:

1. Create an instance of the `LogisticRegression` model.
2. Train the model using the classification training data.
3. Make predictions on the test data.
4. Evaluate the model using `accuracy_score`.

In [4]:
```
# --- ENTER YOUR CODE HERE ---

# 1. Create the model instance
# max_iter=1000 allows more iterations for the model to converge
log_model = LogisticRegression(max_iter=1000)

# 2. Train the model
# The model learns patterns associated with passenger survival
log_model.fit(X_train_cls, y_train_cls)

# 3. Make predictions
# Predict whether each passenger in the test set survived
y_pred_cls = log_model.predict(X_test_cls)

# 4. Evaluate the model
# Accuracy measures the proportion of correct predictions
accuracy = accuracy_score(y_test_cls, y_pred_cls)

# Print the accuracy as a percentage
print(f"Accuracy for Survival Prediction: {accuracy:.2%}")
```

```
Accuracy for Survival Prediction: 74.83%
```

📝 Knowledge Check

Instructions: Answer the following questions in this markdown cell.

1. **In your own words, what is the key difference between a regression problem and a classification problem?**
2. **The `LinearRegression` model has an attribute called `.coef_`. After you train the model, print `lr_model.coef_`. What do these numbers represent?**
3. **Why did we use `mean_squared_error` to evaluate the regression model but `accuracy_score` for the classification model?** Why wouldn't accuracy be a good metric for the fare prediction task?

[ENTER YOUR ANSWERS HERE]

1. Regression is used to predict numerical values such as how much each passenger paid for their ticket. Classification is used to predict if something falls into a certain category such as if the passengers survived.

2. These numbers give information on how the ticket price will be affected by each feature. The first number shows that depending on the passengers age the ticket price is expected to increase/decrease by this amount. The second number shows how the ticket price will increase/decrease depending on the passengers class.

3. We use mean squared error for regression so we can see how far away the predicted numerical value is from the actual value. Accuracy is a better choice for classification as we can see how many predictions were correct. Accuracy would not be a good choice for predicting the ticket price as even if the ticket price is slightly off it would still be counted as incorrect. Mean squared error gives a better understanding of how accurate your predictions are.